automated support while still leaving the decision-making and implementation to the user, and are also required by other reasoning tasks.

The paper is organized as follows. We first give an overview of foundations upon which our technique is built. Then, we present the syntax and semantics of our Contract Definition Language (CDL). We present two case studies involving modeling with CDL and automatically reasoning about two U.S. Federal statutes, FERPA and HIPAA. We conclude and identify next steps after a discussion of related work.

2 Foundations

In this section we give a short overview of the syntax and intuitive semantics of deductive databases and logic programs, two foundational techniques upon which we build our Contract Definition Language.

2.1 Databases

The *vocabulary* of a database is a collection of object constants, function constants, and relation constants. Each function constant and relation constant has an associated arity, i.e. the number of objects involved in any instance of the corresponding function or relation. A term is either a symbol or a functional term. A functional term is an expression consisting of an n-ary function constant and n terms. In what follows, we write functional terms in traditional mathematical notation - the function followed by its arguments enclosed in parentheses and separated by commas. For example, if `f` is a binary function constant and if `a` and `b` are object constants, then `f(a,a)` and `f(a,b)` and `f(b,a)` and `f(b,b)` are all functional terms. Functional terms can be nested within other functional terms. For example, if `f(a,b)` is a functional term, then so is `f(f(a,b),b)`. A datum is an expression formed from an n-ary relation constant and n terms. We write data in mathematical notation. For example, we might write `parent(art,bob)` to express the fact that Art is the parent of Bob. A dataset is any set of data that can be formed from the vocabulary of a database. Intuitively, we can think of the data in a dataset as the facts that we believe to be true in the world; data that are not in the dataset are assumed to be false.

2.2 Logic Programs

The language of logic programs includes the language of databases but provides additional expressive features. One key difference is the inclusion of a new type of symbol, called a *variable*. Variables allow us to state relationships among objects without explicitly naming those objects. In what follows, we use individual capital letters as variables, e.g. `X`, `Y`, `Z`. In the context of logic programs, a term is defined as an object constant, a variable, or a functional term, i.e. an expression consisting of an n-ary function constant and n simpler terms. An atom in a logic program is analogous to a datum in a database except that the constituent terms may include variables. A *literal* is either an atom or a negation of an atom (i.e. an expression stating that the atom is false). A simple atom is called a *positive* literal, The negation of an atom is called a *negative* literal. In what follows, we write negative literals using the negation sign `~`. For example, if `p(a,b)` is an atom, then `~p(a,b)` denotes the negation of this atom. A *rule* is an expression consisting of a distinguished atom, called the head and a conjunction of zero or more literals, called the *body*. The literals in the body are called *subgoals*. In what follows, we write rules as in the example `r(X,Y) :- p(X,Y) & ~q(Y)`. Here, `r(X,Y)` is the head, `p(X,Y) & ~q(Y)` is the body; and `p(X,Y)` and `~q(Y)` are subgoals.

Semantically, a rule states that the conclusion of the rule is true whenever the conditions are true. For example, the rule above states that `r` is true of any object `X` and any object `Y` if p is true of `X` and `Y` and `q` is not true of `Y`. For example, if we know `p(a,b)` and we know that `q(b)` is false, then, using this rule, we can conclude that `r(a,b)` must be true.

3 Contract Definition Language (CDL)

CDL descriptions are open logic programs. While a traditional logic program is typically used to specify views and constraints on a single database state, CDL descriptions can specify a state-transition system. CDL is expressive enough to define a Turing machine. The declarative syntax and formal semantics of CDL makes CDL descriptions easier to comprehend and maintain. The executability of CDL descriptions makes it superfluous to hard-code contract terms with procedural code as well as makes CDL a promising alternative for defining self-executable contracts such as Ethereum Smart Contracts [6, 10].

The basis for CDL is a conceptualization of contracts in terms of entities, actions, propositions, and parties. Entities include objects relevant to the state of a contract are usually represented by object constants in CDL. In some cases, we use compound terms to refer to entities. Actions are performed by the parties involved in the contract. As with entities, we use object constants or compound terms to refer to primitive actions. Some actions may not be legal in every state. Propositions are conditions that are either true or false in each state of a contract. In CDL, we designate propositions using object constants or compound terms. Parties are the active entities in contracts. Note that, in each state, some of the contract's propositions can be true while others can be false. As actions are performed, some propositions become true and others become false. On each time step, each party has a set of legal actions it can perform and executes some action in this set. In CDL, we usually use object constants (in rare cases compound terms) to refer to parties. In CDL, the meaning of some words in the language is fixed for all contracts (the contract-independent vocabulary) while the meanings of all other words can change from one contract to another (the contract-specific vocabulary).

There are the following contract-independent structural relation constants.

1. `role(r)` means that r is a role in the contract.
2. `base(p)` means that `p` is a base proposition in the contract.
3. `percept(r,p)` means that `p` is a percept for role `r`.
4. `input(r,a)` means that `a` is an action for role `r`.

To these basic structural relations, we add the following relations for talking about steps.

1. `step(s)` means that `s` is a step.
2. `successor(s1,s2)` means that step `s1` comes immediately before step `s2`.
3. `true(p,s)` means that the proposition `p` is true on step `s`.
4. `sees(r,p,s)` means that role `r` sees percept `p` on step `s`.
5. `does(r,a,s)` means that role `r` performs action `a` on step `s`.
6. `legal(r,a,s)` means it is legal for role `r` to play action `a` on step `s`.